

דוח מסכם

מערכת לחישוב תשלומים ונוסחאות דינמיות

מגישה: שירה חורי

טכנולוגיות: SQL Server, .NET(C#), Node.js, React

[דאשבורד חי לצפייה בתוצאות](#)

1. מבוא ומהות המשימה

הפרויקט עוסק בבניית מנוע חישוב גמיש למערכת תשלומים חכמה. המטרה היא לאפשר למערכת ללמוד נוסחאות חדשות בזמן ריצה מתוך בסיס הנתונים, מבלי לשנות את קוד התוכנה. האתגר המרכזי היה ביצוע החישובים על נפח גדול של מיליון רשומות, תוך השוואת יעילות בין שלוש טכנולוגיות שונות.

2. פירוט פתרונות החישוב

א. שיטת (Dynamic SQL) SQL Server

בשיטה זו, בסיס הנתונים משמש לא רק לאחסון אלא גם כמנוע החישוב המרכזי.

- רעיון הקוד:** יצרתי פרוצדורה (Stored Procedure) שעוברת נוסחה-נוסחה. עבור כל נוסחה, היא בונה משפט SQL בזמן אמת. לדוגמה, אם הנוסחה היא $a + b$, הפרוצדורה מייצרת פקודה שאומרת: "תעדכן את עמודת התוצאה בערך של עמודת A ועוד עמודת B עבור כל המיליון שורות בבת אחת".
- היתרון:** זהו הפתרון הישיר ביותר. אין צורך להוציא את הנתונים החוצה, ולכן זה חוסך זמן יקר של תקשורת בין השרת לבסיס הנתונים.

ב. שיטת (C#) .NET

כאן העבודה מתבצעת בשכבת השרת, תוך ניצול כוח העיבוד של שפת C# והכלים המובנים של Microsoft.

- רעיון הקוד:** התוכנית שולפת את הנתונים מהטבלה ומכניסה אותם לאובייקט שנקרא `DataTable`. כדי לחשב את הנוסחאות, השתמשתי בפונקציית `Compute`. פונקציה זו מסוגלת לקבל מחרוזת טקסט חופשית ולפרש אותה כביטוי חשבוני המשולב בערכי העמודות בכל שורה.
- התאמת הנוסחאות:** מכיוון שחלק מהנוסחאות נכתבו במקור בסינטקס של SQL, הוספתי שכבת קוד שעוברת על מחרוזת הנוסחה ומבצעת "תרגום" (String Replacement) לפני החישוב. (למשל המרת `IF` ל-`IIF` שמתאים למנוע החישוב של ה-`DataTable`).

ג. שיטת Node.js

שיטה זו רותמת את מנוע ה-V8 המהיר של JavaScript לביצוע משימות עיבוד נתונים בצד השרת.

- **רעיון הקוד:** השרת שולף את מיליון השורות ומפרסר (Parse) אותן כאובייקטים בזיכרון. כדי להריץ את הנוסחאות הדינמיות, השתמשתי בפונקציית ה-`eval` המובנית. המנגנון מקבל את מחרוזת הנוסחה, "מזריק" לתוכה את ערכי המשתנים (a, b, c, d) מהשורה הנוכחית על ידי החלפת טקסט (Replace) ומחזיר את התוצאה הסופית.
- **התאמת הנוסחאות:** בדומה ל-C#, ביצעתי המרות מחרוזת בזמן ריצה כדי להתאים פונקציות SQL (כמו ABS לערך מוחלט, SQRT לשורש או POWER לחזקה) לאובייקט ה-Math של JavaScript. שלב זה היה קריטי כדי לאפשר למנגנון ה-`eval` לפרש את הנוסחה בצורה תקינה ולהחזיר תוצאה מדויקת מבלי לשנות את מקור הנוסחאות בבסיס הנתונים.

3. התמודדות עם עומסים ואופטימיזציה

כדי להריץ מיליון רשומות ב-C# וב-Node.js, ננקטו צעדים מיוחדים למניעת קריסות:

- **ניהול שלמות הנתונים והרצות חוזרות:** בכל הפעלה של אחד ממנועי החישוב המערכת מבצעת ראשית מחיקה ממוקדת של תוצאות קודמות השייכות לאותו מנוע בלבד. פעולה זו מבטיחה שהנתונים המוצגים בדאשבורד מייצגים את ההרצה העדכנית ביותר של כל שפה, מונעת כפילויות בטבלאות הלוגים והתוצאות, ושומרת על השוואה נקייה ואמינה בין הביצועים השונים.
- **עיבוד במנות (Batching of 100k):** גם ב-C# וגם ב-Node.js, טעינה של מיליון רשומות בבת אחת לזיכרון ה-RAM היא מסוכנת. לכן, הקוד פוצל כך שכל סבב מעבד 100,000 רשומות בלבד, שומר אותן ב-DB, ומנקה את הזיכרון לפני המנה הבאה.
- **ניהול זמני המתנה (Command Timeout):** בשתי השפות, חישוב מאסיבי כזה חורג מזמן ברירת המחדל של התקשורת מול ה-SQL. הגדרתי ידנית `CommandTimeout` ארוך מאוד כדי להבטיח שהתהליך יסתיים בהצלחה ולא ייקטע בגלל "Timeout".
- **משתני סביבה (Environment Variables):** לניהול מאובטח וגמיש, פרטי ההתחברות למסד הנתונים (Connection Strings) ופרמטרים של השרת נשמרו בקבצי קונפיגורציה (`env` או `appsettings.json`). זה מאפשר להעביר את המערכת בין סביבת פיתוח לסביבת ייצור בקלות ובבטחה.

4. ארכיטקטורה, ממשק משתמש ופתרונות ענן

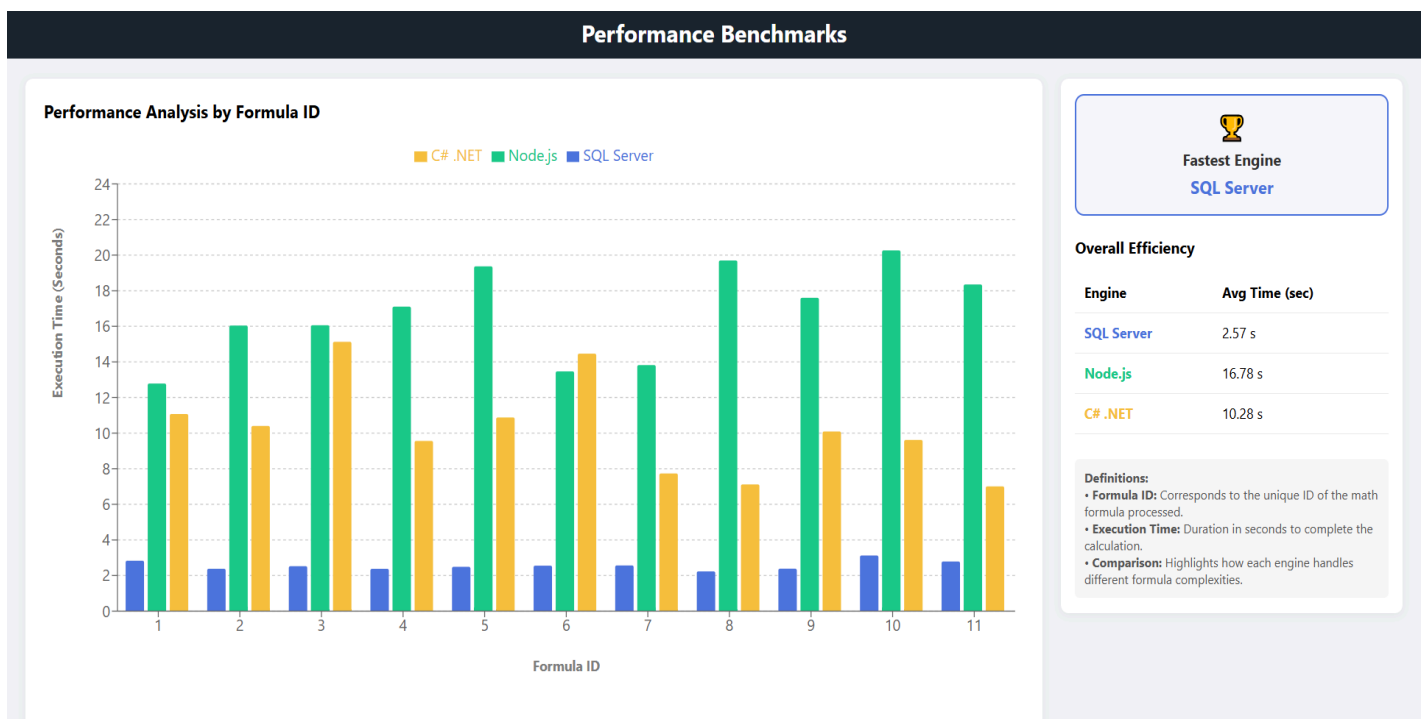
כדי להבטיח שהמערכת תהיה מודולרית, מקצועית וקלה לתחזוקה, ביצעתי הפרדה ברמת הפרויקטים והצגת הנתונים:

- **הפרדת פרויקטים (Architecture):** פרויקט ה-C# חולק לשניים: **מנוע החישוב** (Console Application) האחראי על העבודה ה"כבדה" מול מסד הנתונים, ו-**Web API** נפרד שתפקידו הבלעדי הוא להנגיש את התוצאות המוכנות לממשק המשתמש. הפרדה זו מאפשרת לכל חלק במערכת להיות ממוקד במשימה שלו (Separation of Concerns).
- **ממשק משתמש (React Dashboard):** לצורך הצגת התוצאות בצורה חכמה וויזואלית, פותח ממשק משתמש ב-React המתקשר עם ה-API לשליפת זמני הריצה הממוצעים מטבלת ה-`t_log`.

- **תצוגה ויזואלית:** הממשק כולל דאשבורד עם גרף עמודות המשווה בין המנועים השונים (SQL, C#, Node.js) עבור כל נוסחה בנפרד, וכן טבלת סיכום המציגה את ממוצע היעילות הכללי של כל שפה.
- **פתרון לענן (Static Fallback):** מכיוון שהדאשבורד צריך לרוץ בענן ללא גישה ישירה לבסיס הנתונים המקומי שלי, הטמעת מנגנון **JSON Backup**. במידה וה-API אינו זמין, המערכת מזדהה זאת וטוענת אוטומטית נתונים מקובץ JSON סטטי שהופק מההרצות האמיתיות שביצעת. פתרון זה מבטיח שהדוח והגרפים יהיו תמיד זמינים לצפייה והתרשמות, גם ללא שרת פעיל ברקע. ניתן לראות את היישום בפועל [כאן](#).

4. סיכום ומסקנות

לאחר שווידאתי בעזרת סקריפט השוואה שכל שלוש השיטות הגיעו בדיוק לאותן תוצאות מתמטיות, כך נראה מסך הדו"ח שמשווה בין הביצועים:



1. **המנצח המוחלט: SQL Server** ניצח בביצועים ובמהירות באופן משמעותי, כיוון שהחישוב נעשה בתוך מנוע הנתונים ללא צורך בשליפה החוצה.
 2. **השוואה בין שפות השרת: C# הפגינה יתרון ברור על פני Node.js.** היתרון נבע מהיכולת של C# לנהל זיכרון בצורה יעילה יותר ומהאופטימיזציה של מנוע ה-`DataTable.Compute`, שהתמודד עם עומס החישובים מהר יותר מאשר מנגנון ה-Evaluation ב-Node.js.
- המלצה סופית:** במערכת תשלומים מודרנית, השילוב המנצח הוא שימוש בחוזקות של כל שפה:
- **SQL** לביצוע החישובים הכבדים ביותר על מסד הנתונים.
 - **C# או Node.js** כשכבת שרת (Backend) המנהלת את הלוגיקה העסקית והאבטחה.

לעיון מעמיק בגרפים האינטראקטיביים ובנתוני ההשוואה המלאים, ניתן להיכנס [לאתר הפרויקט](#)